

Contenuti

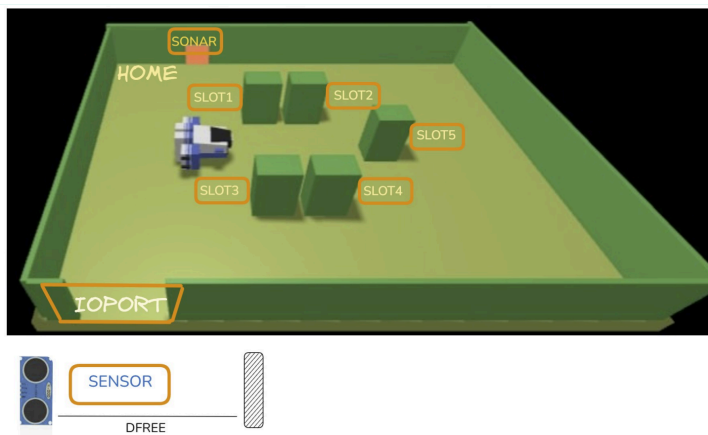
Sprint 0: analisi dei requisiti, descrizione e modellazione dei componenti, piano di test e goal per Sprint 1.

Requisiti

A Maritime Cargo shipping company (from now on, simply **company**) intends to automate the operations of load of **containers** in the ship's cargo hold (or simply **hold**). To this end, the company plans to employ a *Differential Drive Robot* (from now, called **cargorobot**). The **hold** is a rectangular, flat area with an Input/Output port (**IOPort**). The area provides 4 *slots* to store the containers and a slot named **slot5**.

In the picture below:

- The **slots1-4** depict the hold areas reserved to store one container each
- The **slot5** depicts an area, where the cargorobot must temporarily store a container, before to place it in one of the slots1-4. During temporary storage, a 'marker' device labels the container with an identification barcode and signals when this marking activity is completed
- The **IOPort** is a device with a **pushbutton** and a **display**. The pushbutton is pressed by the customer in order to to send a request to load a container on the cargo. The display is used to show the answer to the request and to show the current state of the hold
- The **sensor** associated to the IOPort is a device (a sonar) used to detect the presence of a container, when it measures a distance D , such that $D < D_{FREE}/2$, during a reasonable time (e.g. 3 secs)



Analisi del Problema

Hold

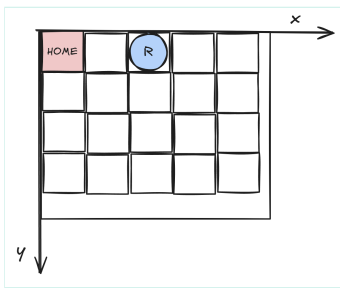
La **hold** è lo spazio di lavoro del cargorobot. Usiamo un sistema di coordinate basato sulla dimensione del robot: definiamolo come cerchio con raggio r , la stanza di base B e altezza H .

```
int hold[ ][ ]
int width = B/2r    // width*2r < B < width*(2r+1)
int height = H/2r   // height*2r < H < height*(2r+1)
```

La posizione **HOME** è definita come `hold[0][0]`. Il committente fornisce la mappa con i seguenti punti di interesse:

```
hold[1][2] = slot1
hold[1][3] = slot2
hold[3][2] = slot3
hold[3][3] = slot4
```

```
hold[2][4] = slot5    // slot temporaneo per marcatura
hold[0][5] = IOPORT   // posizione della IOPort nella griglia
```



Natura software: la hold è modellata come **POJO**, in quanto entità passiva che contiene altri POJO. La logica di orchestrazione risiede nel **cargoservice**.

Slot

Gli slot sono celle della hold che contengono al massimo 1 container. Distinguiamo due tipologie:

- **slot1-4:** slot **permanenti**. Destinazione finale del container.
- **slot5:** slot **transitorio**. Il container vi sosta per la marcatura (barcode). Un dispositivo "marker" applica l'etichetta e segnala il completamento al sistema.

Modelliamo gli slot come **POJO**:

```
bool occupato    // false = libero, true = container presente
```

Natura software: gli slot sono modellati come **POJO** in quanto entità passive.

IOPort

La **IOPort** è il punto di ingresso/uscita dei container nella hold.

Logicamente, la IOPort è composta da:

- **pushbutton:** pulsante fisico. Premuto dal cliente, invia una **richiesta di carico** al cargoservice.
- **display:** mostra risposta e stato della hold.
- **sensor:** rileva la distanza D di un ostacolo.
- **led:** notifica visivamente quando il sistema è engaged.

Natura software: dal punto di vista fisico, è una posizione speciale nella hold (hold[0][5]).

Al livello logico invece, è l'**ATTORE ioservice** (contesto **ctx_io**) che orchestra le periferiche fisiche **pushbutton**, **display** e **led**, che sono **ATTORI**.

Il **sensor**, pur fisicamente associato alla IOPort, è gestito da **sensorservice** in un contesto dedicato (**ctxsensor**).

Sensor

Il **sensor** è un componente **attivo** che rileva la presenza o l'assenza di un container presso la IOPort, misurando una distanza D.

Data una distanza D_{FREE}, valgono le seguenti proprietà:

```
D < DFREE/2          = container presente, IOPort occupata
DFREE/2 <= D <= DFREE = misurazione valida, IOPort libera
D > DFREE per più di 3 sec = sistema entra in Out-of-Service
```

Emette periodicamente verso **sensorservice** (interno a **ctxsensor**):

```
Dispatch sensordata : sensordata(D) // distanza grezza misurata
```

Natura software modellato come **ATTORE** in quanto componente **attivo**. I dati grezzi non escono mai da **ctxsensor**:

è **sensorservice** a tradurli in messaggi semantici per cargoservice.

Led

Il **led** è un dispositivo di notifica visiva. I suoi comportamenti:

- **BLINK**: stato **ENGAGED**
- **OFF**: stati **DISENGAGED/OUT OF ORDER**

Natura software modellato come **ATTORE** in quanto componente **attivo** che risiede su un nodo remoto, ad esempio Raspberry Pi.

Il core business

Il **cargoservice** è il macro-componente che implementa il core-business e Orchestra tutti gli altri componenti, modellato quindi come **ATTORE**.

La logica di business principale è composta da:

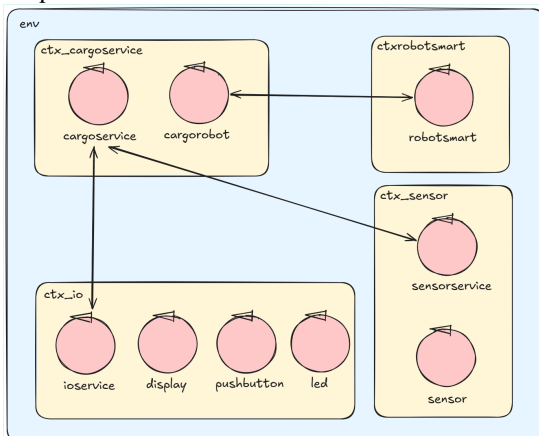
- dove e quando spostare il cargorobot
- accettare o rifiutare una richiesta di carico
- quale slot libero associare alla richiesta che si sta servendo
- mandare out of order il sistema quando si accorge di un malfunzionamento del sensor

Architettura Logica del Sistema

L'architettura logica è descritta nell'immagine. Abbiamo deciso di creare l'attore **cargorobot** per astrarre il dominio di robotsmart: cargoservice ha lo scopo di comunicare le posizioni astratte come slot1-5, home, ioport senza conoscere il sistema di coordinate. Inoltre, se si vuole utilizzare un robot reale basterà modificare solo le chiamate di cargorobot senza cambiare la logica di business di cargoservice.

Il **ioservice** è un attore fondamentale per comunicare con le periferiche hardware (pushbutton, display, led).

Il **sensorservice** (contesto dedicato **ctx_sensor**, insieme al **sensor**) si occupa di gestire il sensor e inviare al cargoservice solo gli eventi generati dal sensor; in questo modo cargoservice riceve solo le informazioni utili (ad esempio quando l'utente deposita il container), al posto di inviare i dati grezzi della distanza D. In questo modo abbiamo una divisione netta dei ruoli e dei domini.



I messaggi principali scambiati tra/nei contesti del sistema sono:

cargoservice (ctx_cargoservice) ↔ ioservice (ctx_io)

- Richieste di carico: il pushbutton notifica ioservice, che inoltra a cargoservice la richiesta di loading:

```
Request  load_request  : load_request(X)
Reply    load_accepted : load_accepted(SLOT)   for load_request //1 | 2 | 3 | 4
Reply    load_rejected : load_rejected(REASON) for load_request //full | out_of_service
Reply    retrylater    : retrylater(X)         for load_request
```

- Comandi da cargoservice verso ioservice:

```
Dispatch setDisplay : setDisplay(MSG) //testo sul display
Dispatch setLed : setLed(STATE) //off | blink
```

cargoservice (ctx_cargoservice) ↔ sensorservice (ctx_sensor)

- Verifica **sincrona** (request-reply) della presenza di un container all'IOPort. Necessaria perché **containerIn** è un dispatch **fire and forget**: se il container è stato depositato **prima** della load_request, il dispatch è già stato emesso (e perso) e cargoservice non può sapere se il container c'è:

```
Request check_container : check_container(X)
Reply container_present : container_present(X) for check_container
Reply container_absent : container_absent(X) for check_container
```

- Notifiche **asincrone** (fire and forget) da sensorservice. Se la verifica sincrona risponde **container_absent**, cargoservice attende il dispatch **containerIn** entro 30s:

```
Dispatch containerIn : containerIn(X) //container rilevato all'IOPort
Dispatch containerOut : containerOut(X) //container rimosso
Dispatch sensorError : sensorError(X) //sensor guasto: outOfService
Dispatch sensorReady : sensorReady(X) //sensor ok: recovery da outOfService
```

sensor → sensorservice (interno a ctx_sensor)

- Dati grezzi, mai inoltrati a cargoservice:

```
Dispatch sensordata : sensordata(D) //distanza misurata, periodica
```

cargoservice ↔ cargorobot (interno a ctx_cargoservice)

- Movimentazione con destinazione logica (nessuna coordinata):

```
Request do_move : do_move(DEST) //home|slot1..5|ioport
Reply move_done : move_done(ARG) for do_move
Reply move_failed : move_failed(CAUSE) for do_move
```

cargorobot (ctx_cargoservice) ↔ robotsmart (ctxrobotsmart)

- Pianificazione ed esecuzione del percorso:

```
Request buildPlan : buildPlan(PX,PY,TX,TY)
Reply buildPlanDone : buildPlanDone(PPLAN) for buildPlan
Request doplan : doplan(PPLAN, STEPTIME)
Reply doplandone : doplandone(ARG) for doplan
Reply doplanfailed : doplanfailed(PLANTODO) for doplan
```

Componenti da Sviluppare

Da sviluppare

- cargoservice**: Sprint 1 (core business, gestione stati)
- cargorobot**: Sprint 1 (astrazione movimentazione robotsmart)
- hold**: Sprint 1 (gestione mappa e slot)
- sensorservice**: Sprint 2 (attività del sensore a cargoservice)
- sensor**: Sprint 2 (rilevazione distanza)
- ioservice**: Sprint 3 (gestione io hw)
- pushbutton**: Sprint 3 (input virtuale/hw)
- display**: Sprint 3 (output verso utente / webGUI)
- led**: Sprint 3 (notifica visiva virtuale/hw)

Forniti dal committente

- robotsmart26** (sistema QAK)
- WEnv** (VirtualRobot26, Docker)

Test Plans

I test sono scritti in Java (JUnit). Sono dei test preliminari, verranno implementati nello Sprint1 e potrebbero cambiare sintatticamente.

Prima di ogni test il sistema è avviato e **cargoservice** è raggiungibile.

sensorservice è simulato: i dispatch **containerIn**, **sensorError**, **sensorReady** vengono iniettati direttamente dal tester, mentre uno **stub** di sensor**service** risponde alle **check_container**.

T01: Richiesta accettata - ENGAGED

Precondizione: sistema in DISENGAGED, almeno uno slot1-4 libero.

Atteso: cargoservice risponde **load_accepted(slotN)** e passa a ENGAGED.

```
@Test
public void testT01() throws Exception {
    String req = CommUtils.buildRequest("tester", "load_request", "load_request(client1)", "cargoservice").toString();
    String resp = conn.request(req);

    assertTrue("T01: load_accepted con slot assegnato", resp.contains("load_accepted"));
}
```

T02: Container depositato - normale operazione - DISENGAGED (normale flow di operazione)

Precondizione: sistema in ENGAGED (T01 eseguito), IOPort libera al momento della richiesta.

Atteso: dopo **containerIn** da sensor**service**, cargorobot esegue il ciclo IOPORT - SLOT5 - slotN, il sistema torna in DISENGAGED con lo slot occupato.

```
@Test
public void testT02() throws Exception {

    //porta il sistema in ENGAGED
    String req = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    conn.request(req);

    //simula sensorservice: container rilevato all'IOPort (dispatch asincrono)
    conn.forward(CommUtils.buildDispatch("tester", "containerIn", "containerIn(x)", "cargoservice").toString());

    CommUtils.delay(8000); //attendo finisca la normale operazione

    //una nuova richiesta deve essere accettata (sistema tornato in DISENGAGED)
    String req2 = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    String resp2 = conn.request(req2);

    assertTrue("T02: sistema tornato in DISENGAGED dopo carico", resp2.contains("load_accepted"));
}
```

T03: Timeout in ENGAGED - DISENGAGED - slot liberato

Precondizione: sistema in ENGAGED, un solo slot libero.

Atteso: allo scadere dei 30s senza **containerIn**, il sistema torna in DISENGAGED con lo slot liberato (prima riservato) e risponde **retrylater** alla richiesta originale.

```
@Test
public void testT03() throws Exception {
    String req = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    String resp = conn.request(req);
    assertTrue("T03 setup: richiesta accettata", resp.contains("load_accepted"));

    //non piazza un container, attendo scadenza timer (5 secondi per testare)
    CommUtils.delay(6000);

    //lo slot deve essere stato liberato
    String req2 = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    String resp2 = conn.request(req2);

    assertTrue("T03: slot liberato dopo timeout", resp2.contains("load_accepted"));
}
```

T04: Hold piena - load_rejected

Precondizione: tutti gli slot1-4 occupati.

Atteso: cargoservice risponde `load_rejected`.

```
@Test
public void testT04() throws Exception {

    //invio richiesta di carico
    String req = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    String resp = conn.request(req);

    assertTrue("T04: hold piena - load_rejected", resp.contains("load_rejected"));
}
```

T05: Sensor guasto - Out of Service - retrylater

Precondizione: sistema in DISENGAGED.

Atteso: dopo `sensorError` da sensorservice il sistema entra in OUT_OF_SERVICE, le richieste successive ricevono `retrylater(out_of_service)`.

```
@Test
public void testT05() throws Exception {
    //simulo sensor guasto
    conn.forward(CommUtils.buildDispatch("tester", "sensorError", "sensorError(x)", "cargoservice").toString());
    CommUtils.delay(300);

    //invio richiesta di carico
    String req = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    String resp = conn.request(req);

    assertTrue("T05: out_of_service - retrylater", resp.contains("retrylater"));
}
```

T06: Recovery da Out-of-Service

Precondizione: sistema in OUT_OF_SERVICE (T05 eseguito).

Atteso: dopo `sensorReady` da sensorservice il sistema torna in DISENGAGED e accetta nuove richieste.

```
@Test
public void testT06() throws Exception {
    //porto il sistema in OUT_OF_SERVICE
    conn.forward(CommUtils.buildDispatch("tester", "sensorError", "sensorError(x)", "cargoservice").toString());
    CommUtils.delay(300);

    //simulo fine guasto
    conn.forward(CommUtils.buildDispatch("tester", "sensorReady", "sensorReady(x)", "cargoservice").toString());
    CommUtils.delay(300);

    //invio richiesta di carico
    String req = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    String resp = conn.request(req);

    assertTrue("T06: recovery - sistema torna in DISENGAGED", resp.contains("load_accepted"));
}
```

T07: Container già presente all'IOPort - carico immediato

Precondizione: container depositato *prima* della load_request (il dispatch `containerIn` è fire and forget ed è già andato perso).

Atteso: alla load_request, cargoservice interroga sensorservice con `check_container`, lo stub risponde `container_present` e il ciclo di movimentazione parte subito, senza attendere i 30s.

```
@Test
public void testT07_ContainerAlreadyAtIoport() throws Exception {
    //setto lo stub del sensorservice in stato "container presente"
    sensorStub.setContainerStatus(true);
}
```

```

//invio richiesta di carico
String req = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
String resp = conn.request(req);
assertTrue("T07 setup: richiesta accettata", resp.contains("load_accepted"));

CommUtils.delay(8000); //attendo finisca l'operazione

//sistema tornato in DISENGAGED: nuova richiesta accettata
sensorStub.setContainerStatus(false);
String req2 = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
String resp2 = conn.request(req2);

assertTrue("T07: carico avviato con container già presente", resp2.contains("load_accepted"));
}

```

Goal Sprint 1

Lo **Sprint 1** ha come obiettivo la realizzazione del core business, per ora senza dipendere da IOPort e dal sensore.

Sottoinsieme di Requisiti (goal Sprint 1)

- Implementare **cargoservice**
- Implementare **cargorobot**
- Gestire la riserva degli slot e il ciclo di movimentazione
- Gestire la verifica sincrona **check_container** verso lo stub di sensorservice
- Gestire il timeout di 30 secondi per il deposito (attesa asincrona di **containerIn** da sensorservice)
- Sensor simulato (stub di sensorservice)
- Hold e slot (POJO)

Test da superare in Sprint 1

- **T01**: Richiesta accettata
- **T02**: Container depositato in tempo
- **T03**: Timeout / DISENGAGED
- **T04**: Hold piena
- **T05**: Sensor guasto / OUT_OF_SERVICE
- **T06**: Recovery da OUT_OF_SERVICE
- **T07**: Container già presente all'IOPort (check sincrono)

Componenti prodotti in Sprint 1

- **cargoservice**
- **cargorobot**
- **hold**

Piano di lavoro

- Prevediamo al massimo 20 ore/uomo, su una settimana di lavoro.
- Francesco si occuperà principalmente dell'implementazione QAK.
- Andrea si occuperà principalmente dei Test e stesura del report.
- Entrambi ci aggiorniamo giornalmente sul lavoro svolto, studiamo e valutiamo i rispettivi lavori.

By Andrea Gazzola



and Francesco Roffia



[GIT repo:](#)

